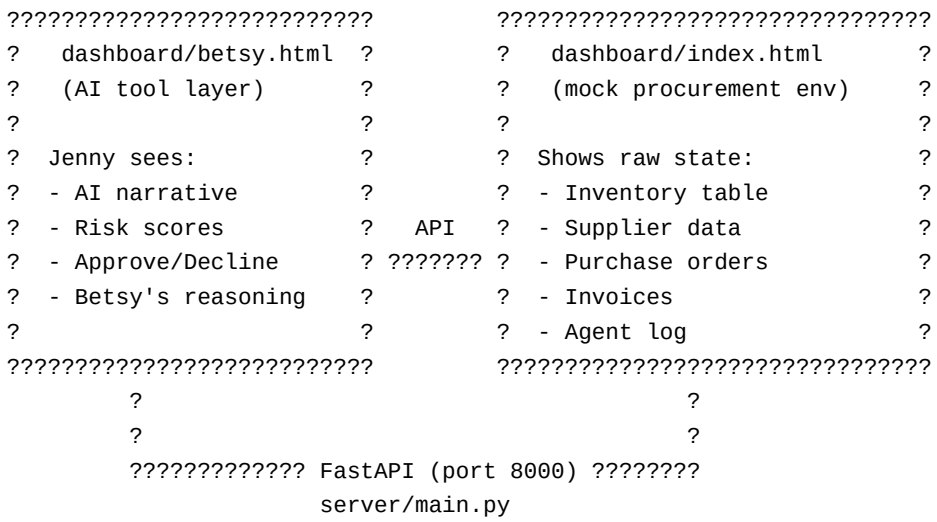
Betsy -- API Control Layer

How agent decisions become visible changes in the procurement environment. Use this document as evidence for DL-03 (data layers / HITL design).

The Two Surfaces



Both surfaces read from and write to the same FastAPI server. An action taken in betsy.html is immediately visible in index.html on the next refresh.

Agent Actions -> API Calls

Agent Decision	API Call	What changes in the environment
generate_po (auto-approved)	POST /api/purchase-orders	New row in Purchase Orders tab of index.h
generate_po (requires human)	POST /api/approvals	Appears in Betsy's "Needs your OK" section
Human approves in betsy.html	POST /api/approvals/{id}/approve -> triggers POST /api/purchase-orc	
Human declines in betsy.html	POST /api/approvals/{id}/reject	Item logged, no PO created
flag_duplicate	POST /api/agent-log + invoice status update	Duplicate badge appears in Invoice t
flag_for_approval (price spike)	POST /api/approvals	Appears in Betsy's "Needs your OK" section
escalate (supplier OOS)	POST /api/agent-log	Entry appears in Agent Log tab of index.html
Any agent run	POST /api/agent-log	Agent Log tab updates, Betsy narrative refreshes

---

Data Flow Per Action

generate_po (auto)

Agent decide node

```
  -> POST /api/purchase-orders
      { supplier_id, sku_id, quantity, unit_price, reason: "stockout_risk", requested_by: "betsy-agent"
  -> Server creates PO with status: "pending_approval"
  -> index.html Purchase Orders tab shows new row
  -> betsy.html AI narrative updates: "I ordered X from Y"
```

generate_po (requires human)

```
Agent decide node  (confidence < threshold OR amount > MAX_AUTO_USD)
  -> POST /api/approvals
      { action: "generate_po", sku_id, supplier_id, po_total, reasoning, payload }
  -> betsy.html "Needs your OK" section shows card
  -> Jenny clicks Approve
      -> POST /api/approvals/{id}/approve
      -> Server triggers POST /api/purchase-orders with stored payload
      -> index.html Purchase Orders tab shows new PO row
      -> betsy.html approval card disappears
```

flag_duplicate

```
Agent invoice_auditor
  -> POST /api/agent-log
      { trigger: "duplicate_invoice", decision: "flag_duplicate", ... }
  -> index.html Agent Log tab shows entry
  -> index.html Invoice tab: duplicate pair highlighted in red
  -> betsy.html AI narrative: "I flagged a possible duplicate invoice"
```

API Endpoints Reference

Read (both surfaces poll these)

```
Endpoint  |  Returns
--------------------
GET /api/inventory  |  All SKUs with current stock levels
-----------------------------------------------------------
GET /api/suppliers  |  All suppliers with catalog + reliability scores
-----------------------------------------------------------------------
GET /api/purchase-orders  |  All POs (all statuses)
-------------------------------------------------------------------------
GET /api/invoices  |  All invoices
-------------------------------------------------------------------------
GET /api/invoices/duplicates  |  Detected duplicate pairs
-------------------------------------------------------------------------
GET /api/agent-log  |  All agent run entries
-------------------------------------------------------------------------
GET /api/approvals  |  Pending human approval items
-------------------------------------------------------------------------
GET /api/scenario  |  Currently active scenario
-------------------------------------------------------------------------
```

Write (agent or betsy.html triggers these)

```
Endpoint  |  Used by
--------------------
```

```
POST /api/purchase-orders  |  Agent (auto-approve) or server (on human approve)
-------------------------------------------------------------------------------
POST /api/agent-log  |  Agent audit node after every run
-------------------------------------------------------------------------------
POST /api/approvals  |  Agent when requires_human=True
-------------------------------------------------------------------------------
POST /api/approvals/{id}/approve  |  betsy.html approve button
-------------------------------------------------------------------------------
POST /api/approvals/{id}/reject  |  betsy.html decline button
-------------------------------------------------------------------------------
POST /api/scenario/{name}  |  index.html scenario injection (demo only)
-------------------------------------------------------------------------------
POST /api/scenario/reset  |  index.html reset button (demo only)
-------------------------------------------------------------------------------
```

Why This Separation Matters

The AI tool (betsy.html) and the environment (index.html) share the same data layer but serve different audiences:

- index.html -- developers, demo, scenario injection, raw data inspection
- betsy.html -- end users (Jenny), plain English, action-oriented, no raw IDs

This separation is the "AI as a layer" principle in practice: - The environment doesn't know or care that Betsy exists - Betsy reads from and writes to the same APIs any human operator would use - If Betsy is turned off, the environment still works normally - Betsy's footprint is: `POST /api/agent-log` entries + `POST /api/purchase-orders` (when auto-approved)

Scenario Injection -> What Betsy Sees

When a scenario is injected via index.html, the environment state changes immediately. On Betsy's next refresh (every 5s), the AI narrative and risk scores update automatically.

```
Scenario injected  |  What changes in betsy.html
-----------------------------------------------
stockout_warning  |  SKU-003 appears ? Critical in inventory table, narrative mentions it
-------------------------------------------------------------------------------
price_spike  |  Copper Wire shows ? Watch + "Flagged" in action column
-------------------------------------------------------------------------------
duplicate_invoice  |  Narrative flags the duplicate, agent log shows the entry
-------------------------------------------------------------------------------
supplier_oos  |  Affected supplier shows as unavailable in supplier table
-------------------------------------------------------------------------------
```

Use as evidence for: DL-03 (data layer design), DL-04 (HITL method), user_requirements.md (F3, F4)